

28 Aug 2026

National Science Foundation
Program Directors, Pathways to Enable Open-Source Ecosystems (PESOSE), NSF 26-506

Subject: Letter of Collaboration — "PESOSE: Track 3: RedTwin: AI-Enabled Digital Twins for Continuous Red Teaming of Open-Source Ecosystems" (Track 3)

Dear Program Directors,

My name is Gabriele Santomaggio. I work on distributed systems at Broadcom, where I am a member of the RabbitMQ team. RabbitMQ is a widely deployed open-source message broker, and my work concerns the broker itself, its protocol implementations, and its client libraries. I write in support of the RedTwin proposal submitted by Prof. Giovanni Vigna and Prof. Christopher Kruegel (UC Santa Barbara) and Prof. Adam Doupe and Prof. Tiffany Bao (Arizona State University).

Relationship to the proposing team. I have no relationship with the proposing team. I have not collaborated with, co-authored with, studied under, or advised any of the PIs, I have no financial interest in the project, and I would receive no funding from it. The team shared the technical plan with me because RabbitMQ is one of the open-source components the project intends to analyze, and because it appears in the proposal's motivating scenario. The assessment below is my own.

Assessment of the proposed technical approach. What convinced me is not the ambition of the proposal but the specific choices it makes, three of which address problems I recognize from maintaining a broker that people run in the middle of critical systems.

The first is the decision to separate passive observation of the production system from aggressive analysis performed only on the twin. This matters more for a message broker than it might appear. The deployments where the security questions are most consequential are exactly the ones where no operator will ever authorize fuzzing a live broker or flooding it with malformed frames: the broker sits between subsystems that must keep working, and an availability incident caused by a security test is indistinguishable, to the people affected, from an outage caused by an attacker. Confining destructive work to a twin that can be reset at no cost removes that objection.

The second is that the proposal treats the fidelity of the twin as a research problem to be measured rather than an assumption to be asserted. Collapsing functionally identical components into a single representative with a multiplicity annotation is the right way to make replication tractable at real scale, and it is also precisely where a clustered, stateful system can be misrepresented: a RabbitMQ cluster is not a set of interchangeable replicas behind a load balancer, and node membership, quorum, the behavior of queues under network partition, and the trust relationships among nodes are security-relevant properties that a careless abstraction would erase along with the redundancy. The proposal is not naive about this. Logic-based invariants over reachability and topology, attack-graph preservation analysis intended to show that the twin's attack graph is a sound abstraction of the original's, and a fidelity measure that separates soundness from precision and localizes any loss of precision to a specific abstraction decision give a maintainer something we almost never get today: a stated basis for judging whether a reported finding applies to real deployments, and a way to say which abstraction decision needs refining when it does not. A broker cluster strikes me as a good stress test for that machinery.

The third is the treatment of a discovered defect as an exploitation primitive with explicit preconditions and postconditions that a planner can compose. This matches how risk actually materializes around brokers. The more damaging problems are rarely a single memory error in a protocol parser; they arise from composition — an overly permissive set of virtual-host permissions, a credential shared between a service and the broker, a management interface reachable from a network segment nobody intended, an inter-node channel trusted because it is assumed to be private. Each is unremarkable in isolation, which is why they persist, and each is a step in a chain when combined with the rest of the deployment. The proposal's motivating scenario, in which an attacker who reaches a device gateway pivots through the broker toward a database, is not contrived; it is the shape of the report we find most useful and receive least often.

My contribution to the open-source product. My contribution is ongoing work on RabbitMQ as an open-source project: development and review of the broker and its client libraries, protocol and plugin work, participation in design discussions, and support of the user and contributor community through the project's public channels, talks, and documentation.

What I would hope for if the proposal is funded. My interest is that the results reach the distribution rather than stopping at a publication. Findings in the code (most plausibly in protocol handling and in the interfaces the broker exposes over the network) would be handled through the project's normal security reporting process, and the more durable value would be regression tests that keep a class of defect from reappearing. Findings about deployment and configuration would be at least as valuable, because they point to things we control: safer defaults where compatibility permits, configuration validation and diagnostics that make a dangerous combination visible before it reaches production, and corrections to the documentation and reference deployments that users copy. That is how work of this kind reaches operators who will never read the paper.

I would also expect the exchange to run in both directions, and this is the part I find most interesting. Replicating a broker faithfully depends on details that mostly live with maintainers: which aspects of cluster formation and inter-node communication are security-relevant, how virtual-host and permission structures should be represented, which plugin combinations change the exposed surface, and where a superficially correct replica would diverge from a real deployment under load or partition. Equally, reports of chains that do not reproduce on the original system, and the reasons why, are information only maintainers can supply, and they appear directly useful to the project's fidelity work. Any involvement of mine or of my colleagues would be subject to Broadcom's internal review, but I see value on both sides and would welcome the conversation.

I believe this research could make RabbitMQ and the systems built around it measurably safer, and I am glad to support the proposal.

Sincerely,

Gabriele Santomaggio
Distributed Systems — RabbitMQ Team
Broadcom Inc.
[gabriele.santomaggio@broadcom.com] | [3939731936]